

MLOps

LIME – Shap Values Jenkins – Kubeflow – Airflow

Session 9P

Juan Ramírez
jramirezg@faculty.ie.edu

LIME

LIME

<https://github.com/marcotcr/lime>

☰ README.md

lime

Local Interpretable Model-Agnostic Explanations

build passing launch binder

This project is about explaining what machine learning classifiers (or models) are doing. At the moment, we support explaining individual predictions for text classifiers or classifiers that act on tables (numpy arrays of numerical or categorical data) or images, with a package called lime (short for local interpretable model-agnostic explanations). Lime is based on the work presented in [this paper](#) (bibtex [here](#) for citation). Here is a link to the promo video:

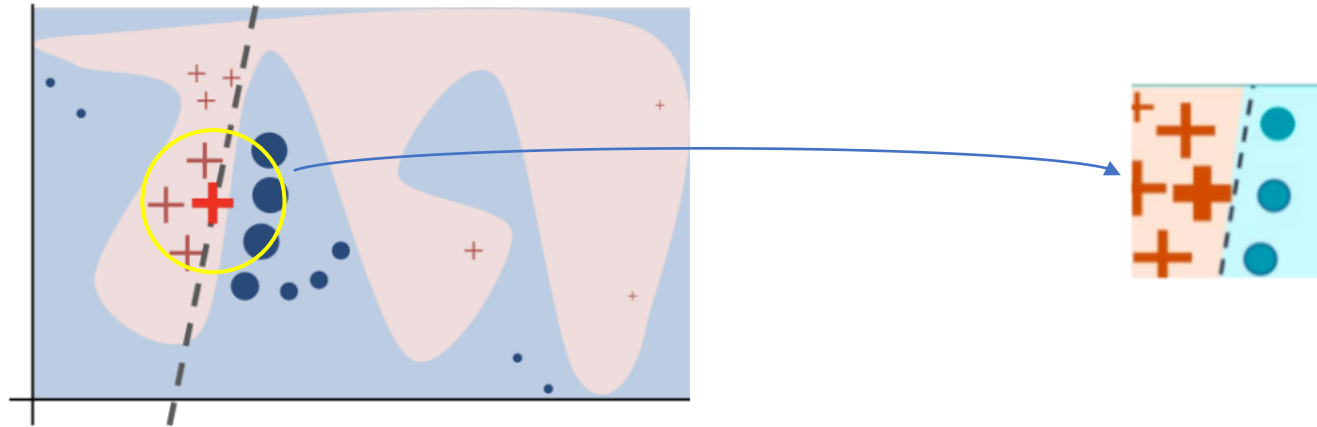


- Local refers to local fidelity - i.e., we want the explanation to really reflect the behaviour of the classifier "around" the instance being predicted.
- This explanation is useless unless it is interpretable - that is, unless a human can make sense of it.
- Lime is able to explain any model without needing to 'peak' into it, so it is model-agnostic -> in other words, LIME assumes that the model is a 'black-box'

How LIME works?



- LIME fits a local interpretable (simpler) model around the instance to be explained



Complex non linear model

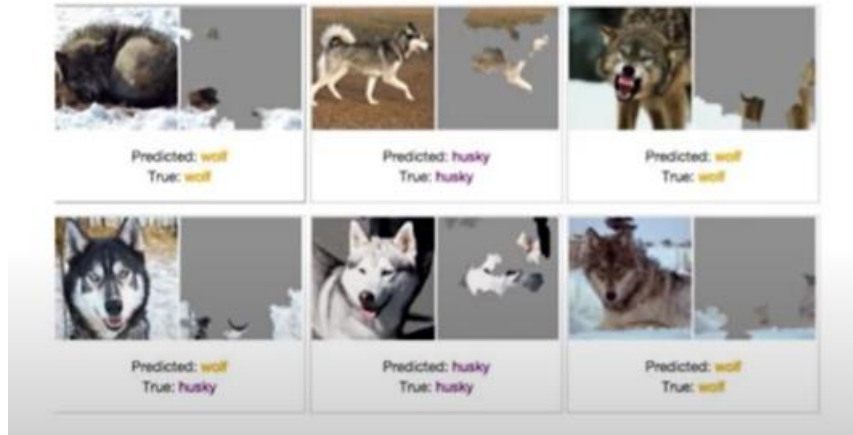
Simpler linear model

1. Perturb the original data points, feed them into the black box model, and then observe the corresponding outputs.
2. The method then weighs those new data points as a function of their proximity to the original point.
3. Pick m features best describing the model outputs from the permuted data
4. Ultimately, it fits a surrogate model such as linear regression on the dataset with variations using those sample weights.
5. Each original data point can then be explained with the newly trained explanation model

Source: Marco Tulio Ribeiro, Sameer Singh, Carlos Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier"

LIME can be applied to image and text analytics

- And it can bring BIG surprises to how your models are performing



Prediction probabilities

atheism	0.58
christian	0.42

atheism

christian

Posting: 0.15
Host: 0.14
NNTP: 0.11
edu: 0.06
have: 0.01
There: 0.01

Text with highlighted words

From: johnchad@triton.unm.edu (jchadwic)
Subject: Another request for Darwin Fish
Organization: University of New Mexico, Albuquerque
Lines: 11

NNTP-Posting-Host: triton.unm.edu

Hello Gang,

There have been some notes recently asking where to obtain the DARWIN fish.

This is the same question I have and I have not seen an answer on the net. If anyone has a contact please post on the net or email me.

How to use LIME

```
# Random Forerst Model
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(random_state=1502, n_estimators=200)

model.fit(X_train, y_train)
```

✓ 8.2s

```
RandomForestRegressor(n_estimators=200, random_state=1502)
```

```
# lime
from lime import lime_tabular
explainer = lime_tabular.LimeTabularExplainer(training_data = np.array(X_train), mode = "regression", feature_names = X_train.columns, categorical_features=[2])
```

✓ 0.8s

```
exp = explainer.explain_instance(data_row = X_test.iloc[0], predict_fn = model.predict)

exp.show_in_notebook(show_table = True)
```

✓ 6.2s

1. Train the model

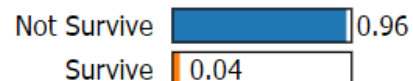
2. Create the explainer

3. Send the instance
that you want to
analyze

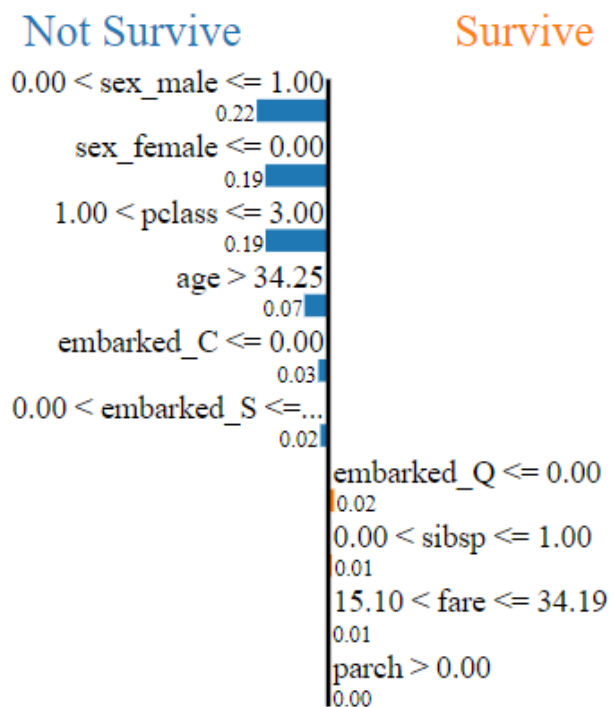
Explanation of one instance

Prediction for the case

Prediction probabilities



Feature contribution to prediction



Feature values

Feature	Value
sex_male	1.00
sex_female	0.00
pclass	3.00
age	36.00
embarked_C	0.00
embarked_S	1.00
embarked_Q	0.00
sibsp	1.00
fare	24.15
parch	1.00

LIME Pros and Cons

Pros

Model agnostic, and different data types (Tabular, Text, Image)

Easy to interpret

Local fidelity measure: evaluate how well the surrogate model approximates the black box predictions in the neighborhood of the data instance

Cons

Interpretability could be unstable since it is based on sampling of data

Local model approximation could be bad -> LIME interpretability unreliable

Consider that the method is still in development

Examples https://github.com/juanmaramirezgarcia/mlops_course_materials/tree/main/11%20Lime

SHAP Values

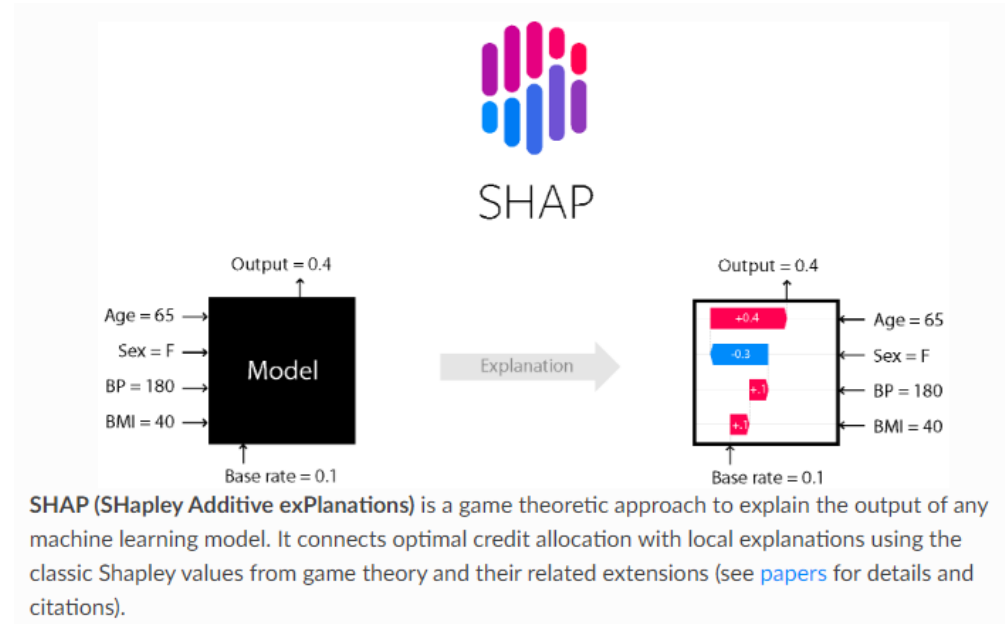
SHAP Values

SHapley Additive exPlanations

<https://shap.readthedocs.io/en/latest/index.html>



- Uses the approach of game theory to explain model predictions: assume that each feature value of the instance is a 'player' in a game where the prediction is the payout
- It starts with some base value for prediction based on prior knowledge and then tries features of data one by one to understand the impact of the introduction of that feature on our base value to make the final prediction.
- It even takes into account orders of feature introduction as well as the interaction between features helping us better understand model performance.
- During this process, it records shap values which will be later used for plotting and explaining predictions



SHAP Explainers

- **AdditiveExplainer** - This explainer is used to explain Generalized Additive Models.
- **BruteForceExplainer** - This explainer uses the brute force approach to find shap values which will try all possible parameter sequence.
- **DeepExplainer** - This explainer is designed for deep learning models created using Keras, TensorFlow, and PyTorch. It's an enhanced version of the DeepLIFT algorithm where we measure conditional expectations of SHAP values based on a number of background samples. It's advisable to keep reasonable samples as background because too many samples will give more accurate results but will take a lot of time to compute SHAP values. Generally, 100 random samples are a good choice.
- **GradientExplainer** - This explainer is used for differentiable models which are based on the concept of expected gradients which itself is an extension of the integrated gradients method.
- **KernelExplainer** - This explainer uses special weighted linear regression to compute the importance of each feature and the same values are used as SHAP values.
- **LinearExplainer** - This explainer is used for linear models available from sklearn. It can account for the relationship between features as well.
- **PartitionExplainer** - This explainer calculates shap values recursively through trying a hierarchy of feature combinations. It can capture the relationship between a group of related features.
- **PermutationExplainer** - This explainer iterates through all permutation of features in both forward and reverses directions. This explainer can take more time if tried with many samples.
- **SamplingExplainer** - This explainer generates shap values based on assumption that features are independent and is an extension of an algorithm proposed in the paper "An Efficient Explanation of Individual Classifications using Game Theory".
- **TreeExplainer** - This explainer is used for models that are based on a tree-like decision tree, random forest, gradient boosting.
- **CoefficientExplainer** - This explainer returns model coefficients as shap values. It does not do any actual shap values calculation.
- **LimeTabularExplainer** - This explainer simply wrap around LimeTabularExplainer from lime library. If you are interested in learning about lime then please feel free to check on our tutorial on the same from references section.
- **MapleExplainer** - This explainer simply wraps MAPLE into shap interface.
- **RandomExplainer** - This explainer simply returns random feature shap values.
- **TreeGainExplainer** - This explainer returns global gain/Gini feature importances for tree models as shap values.
- **TreeMapleExplainer** - This explainer provides a wrapper around tree MAPLE into shap interface.

Linear Explainer

- Select the explainer according to the model. In this example, we selected LinearExplainer as the model was trained with Scikit Learn Linear Regression
- Once calculated the Shap Values, it is possible to check this values for a specific transaction. In this example we selected the first transaction
- The base value is the expected baseline for the model
- The data are the values in the first vector

Trained Linear
Regression Model

Test Data

```
explainer = shap.LinearExplainer(lr_model, val_X)
shap_values = explainer(val_X)
```

```
shap_values[0]
✓ 0.5s

.values =
array([ -5301.08938545, -157189.61630534, -21723.50564051,
        -982.26315043, -128141.47843436,  28219.86599363,
         6013.03882793, -49997.12987594,  63497.74410516,
        -153046.97805976,  19438.29782781, -1826.34719215])

.base_values =
1049127.6307574362

.data =
array([ 3.0000000e+00,  1.5300000e+01,  3.0740000e+03,  3.0000000e+00,
         1.0000000e+00,  2.0000000e+00,  7.2700000e+02,  1.0900000e+02,
         1.9520000e+03, -3.7678730e+01,  1.4501878e+02,  7.9550000e+03])
```

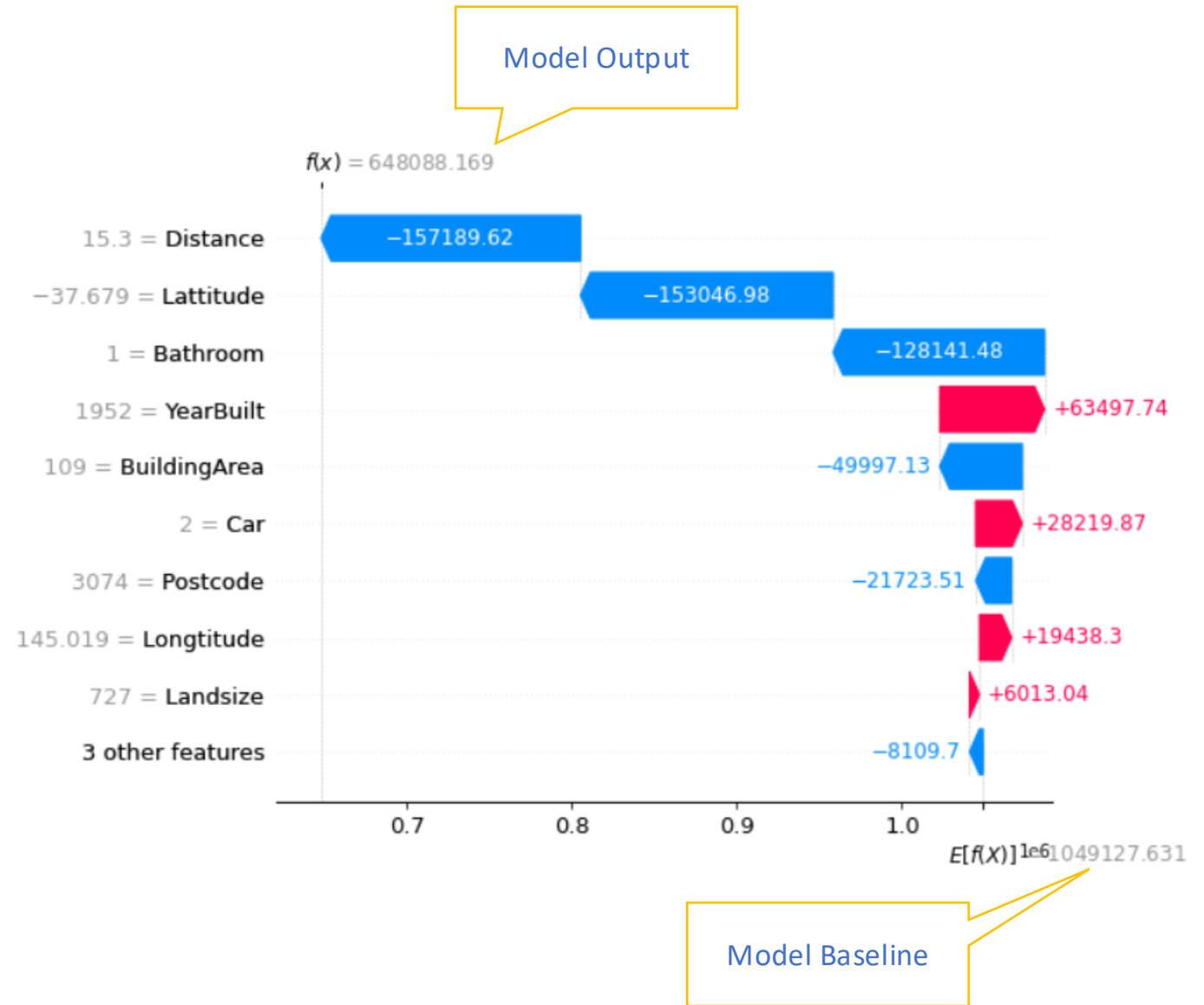
```
val_X.values[0]
✓ 0.1s

array([ 3.0000000e+00,  1.5300000e+01,  3.0740000e+03,  3.0000000e+00,
         1.0000000e+00,  2.0000000e+00,  7.2700000e+02,  1.0900000e+02,
         1.9520000e+03, -3.7678730e+01,  1.4501878e+02,  7.9550000e+03])
```

SHAP Waterfall Plot

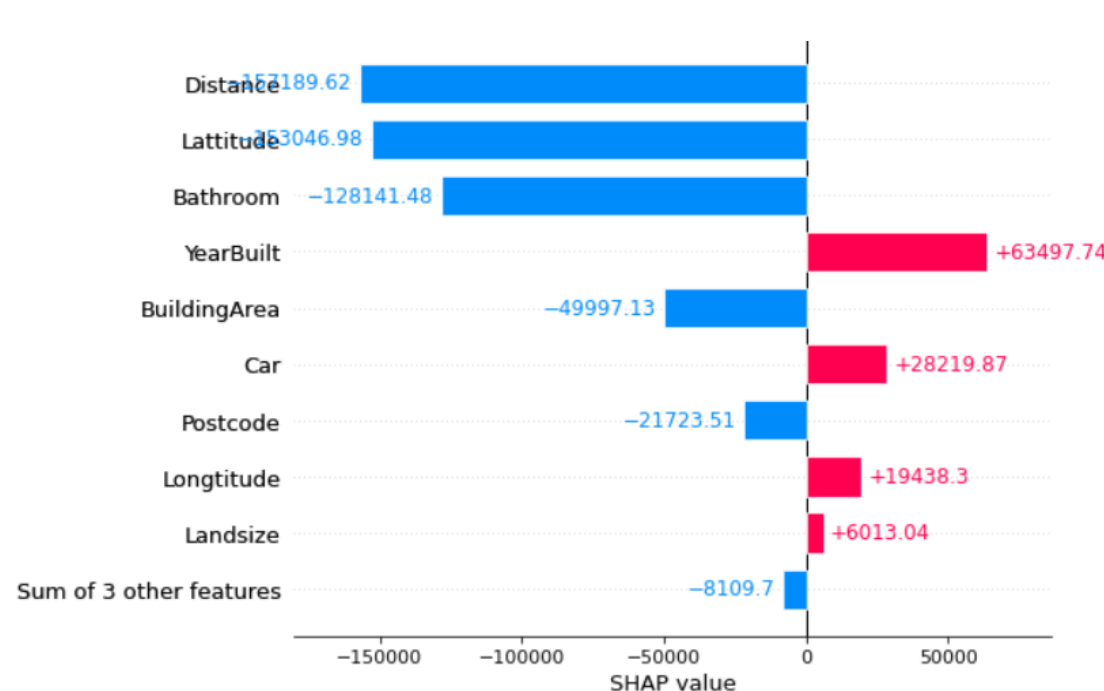
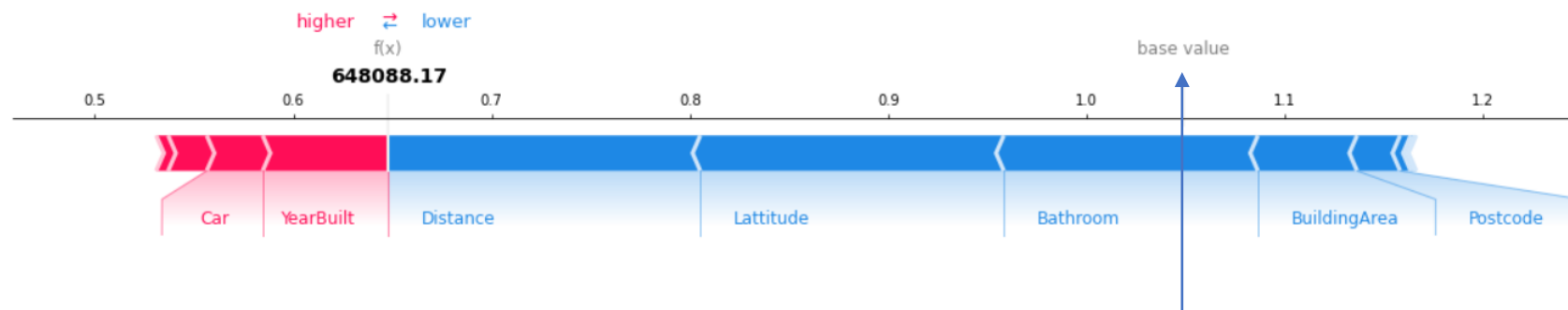
- The easiest way to see this is through a waterfall plot that starts our background prior expectation for the model, and then adds features one at a time until we reach the current model output

```
shap.plots.waterfall(shap_values[0])
```



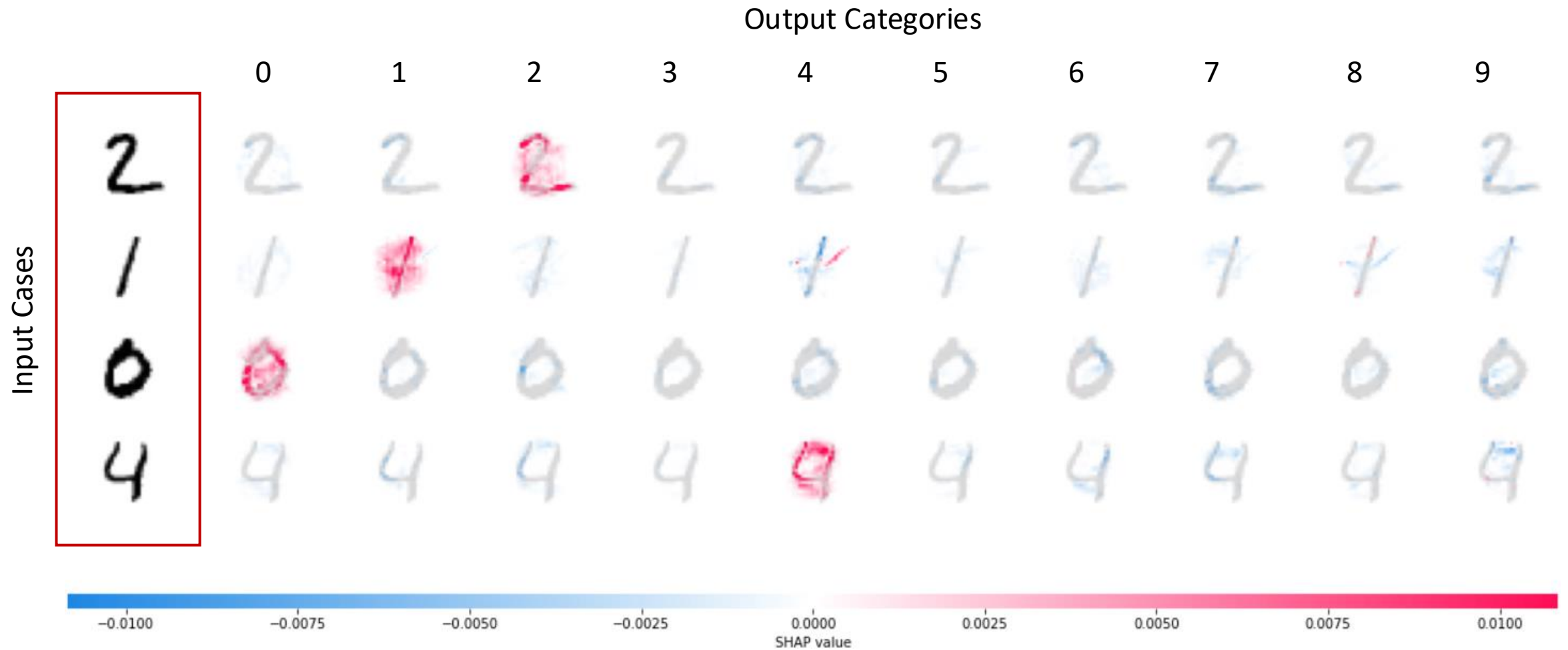
SHAP Force Plot and Bar Plot

```
shap.plots.force(explainer.expected_value, shap_values.values[0], feature_names=val_X.columns, matplotlib=True)
```



```
shap.plots.bar(shap_values[0], max_display=len(val_X.columns), show=True)
```

SHAP Image Plot



Examples https://github.com/juanmaramirezgarcia/mlops_course_materials/tree/main/11%20Shap

Jenkins

Jenkins

<https://www.jenkins.io/>

- Jenkins is the leading open source **continuous integration tool**
- Jenkins was originally developed as the Hudson project. Hudson's creation started in summer of 2004 at Sun Microsystems
- As of today, the Jenkins organization on GitHub had 707 project members



Jenkins

About

Jenkins Weekly 2.347

Jenkins LTS 2.332.3

docker pulls 690M

openssf best practices passing

In a nutshell, Jenkins is the leading open-source automation server. Built with Java, it provides over 1,700 **plugins** to support automating virtually anything, so that humans can spend their time doing things machines cannot.

What to Use Jenkins for and When to Use It

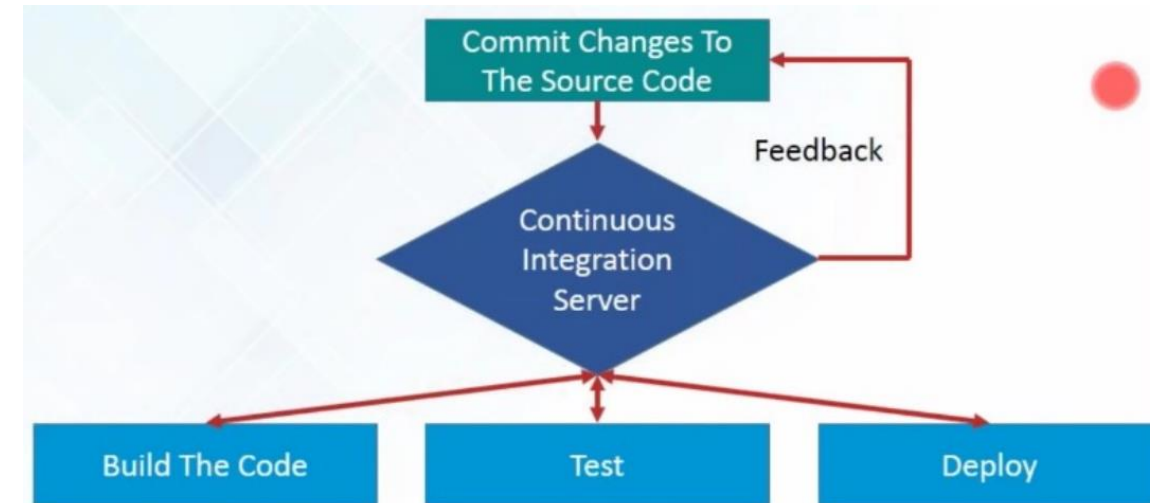
Use Jenkins to automate your development workflow, so you can focus on work that matters most. Jenkins is commonly used for:

- Building projects
- Running tests to detect bugs and other issues as soon as they are introduced
- Static code analysis
- Deployment

Execute repetitive tasks, save time, and optimize your development process with Jenkins.

What is Continuous Integration

- Continuous Integration is a development practice in which developers are required to commit changes to source code in a shared repository
- Every commit is then build and this allows the teams to detect problems early.
- It then bring following benefits to software development:
 1. Catch issues fast
 2. Everyone can see what's happening
 3. Automate the build
 4. Continuous Integration leads to Continuous Deploy



Types of Jenkins Jobs

- Freestyle
 - Freestyle build jobs are general-purpose build jobs, which provides maximum flexibility. It can be used for any type of project.
- Pipeline
 - This project runs the entire software development workflow as code. Instead of creating several jobs for each stage of software development, you can now run the entire workflow as one code
- Multiconfiguration
 - The multiconfiguration project allows you to run the same build job on different environments. It is used for testing an application in different environments.
- Folder
 - This project allows users to create folders to organize and categorize similar jobs in one folder or subfolder.
- GitHub Organization
 - This project scans your entire GitHub organization and creates Pipeline jobs for each repository containing a Jenkinsfile
- Multibranch Pipeline
 - This project type lets you implement different Jenkinsfiles for different branches of the same project.

Pipeline Concepts

- Pipeline: A user-defined block which contains all the stages. It is a key part of declarative pipeline syntax.
- Node: A node is a machine that executes an entire workflow. It is a key part of the scripted pipeline syntax.
- Agent: instructs Jenkins to allocate an executor for the builds. It is defined for an entire pipeline or a specific stage.

Link to example repository

<https://github.com/juanmaramirezgarcia/MLOps-with-Docker-and-Jenkins>

Add New Item

 Jenkins

 1

 2

 Juan Ramírez

 log out

Dashboard ▾

New Item

People

Build History

Project Relationship

Check File Fingerprint

Manage Jenkins

My Views

Open Blue Ocean

Lockable Resources

New View

Build Queue

No builds in the queue.

Build Executor Status

Add description

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration	Fav
		Development	4 days 10 hr #29	4 days 12 hr #28	1 min 49 sec	 

Icon: S M L

Icon legend

 Atom feed for all

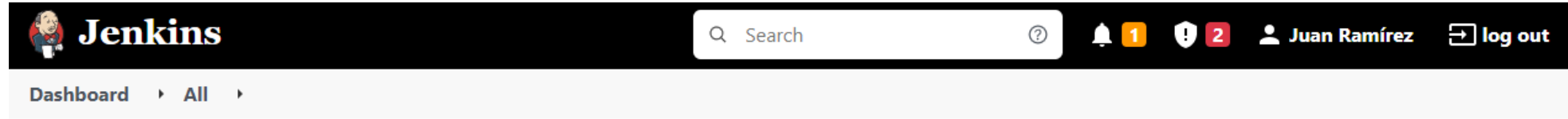
 Atom feed for failures

 Atom feed for just latest builds

MLOps MBDS-PT2025F-1 – Juan Ramírez

21

Enter Item Name



Give project a name
and choose Freestyle
Project as starting
point

Enter an item name

» Required field

**Freestyle project**

This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.

**Pipeline**

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

**Multi-configuration project**

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

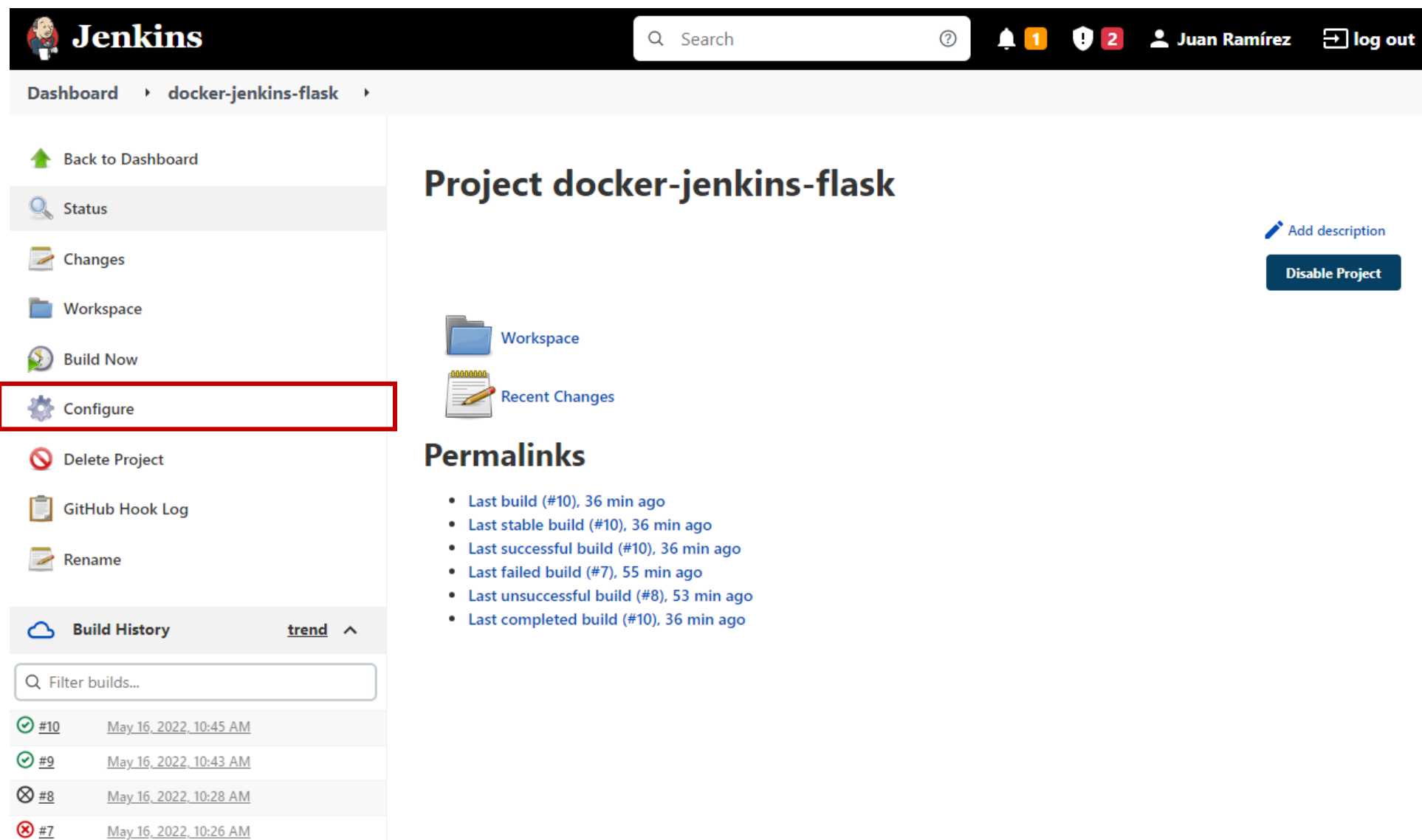
**Folder**

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

**Branch Pipeline**

Creates a set of Pipeline projects according to detected branches in one SCM repository.

Project Configure



The screenshot shows the Jenkins web interface. At the top is a black navigation bar with the Jenkins logo, a search bar, and user information (Juan Ramírez) with a log out button. Below this is a breadcrumb trail: Dashboard > docker-jenkins-flask >. The left sidebar contains a list of actions: Back to Dashboard, Status, Changes, Workspace, Build Now, Configure (highlighted with a red box), Delete Project, GitHub Hook Log, and Rename. The main content area is titled 'Project docker-jenkins-flask' and includes buttons for 'Add description' and 'Disable Project'. It also shows links for 'Workspace' and 'Recent Changes'. Below this is a 'Permalinks' section with a list of build links. At the bottom is a 'Build History' section with a search bar and a table of recent builds.

Jenkins Search ? 1 2 Juan Ramírez log out

Dashboard > docker-jenkins-flask >

Back to Dashboard

Status

Changes

Workspace

Build Now

Configure

Delete Project

GitHub Hook Log

Rename

Build History trend ^

Filter builds...

✓ #10	May 16, 2022, 10:45 AM
✓ #9	May 16, 2022, 10:43 AM
✗ #8	May 16, 2022, 10:28 AM
✗ #7	May 16, 2022, 10:26 AM

Project docker-jenkins-flask

Add description

Disable Project

Workspace

Recent Changes

Permalinks

- Last build (#10), 36 min ago
- Last stable build (#10), 36 min ago
- Last successful build (#10), 36 min ago
- Last failed build (#7), 55 min ago
- Last unsuccessful build (#8), 53 min ago
- Last completed build (#10), 36 min ago

Project Configure Source Code Management

Write repository URL

Dashboard ▾ ▸ **docker-jenkins-flask** ▸

General **Source Code Management** Build Triggers Build Environment Build Post-build Actions

☐ execute concurrent builds if necessary Advanced...

Source Code Management

☐ None
☒ **Git** ?

Repositories ?

Repository URL ?

`https://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@github.com/juanmaramirezgarcia/docker-jenkins-flask.git`

Credentials ?

- none - ▾ Add ▾

Advanced...

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ? X

Project Configure Build Triggers + Build

Select GitHub hook trigger:
Any update in repository
will trigger a new build

Dashboard ▸ docker-jenkins-flask ▸

General Source Code Management **Build Triggers** Build Environment Build Post-build Actions

Build Triggers

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Build Environment

- ☐ Delete workspace before build starts
- ☐ Use secret text(s) or file(s) ?
- ☐ Abort the build if it's stuck
- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published Gradle build scans
- ☐ With Ant ?

Build

Execute Windows batch command ?

Command

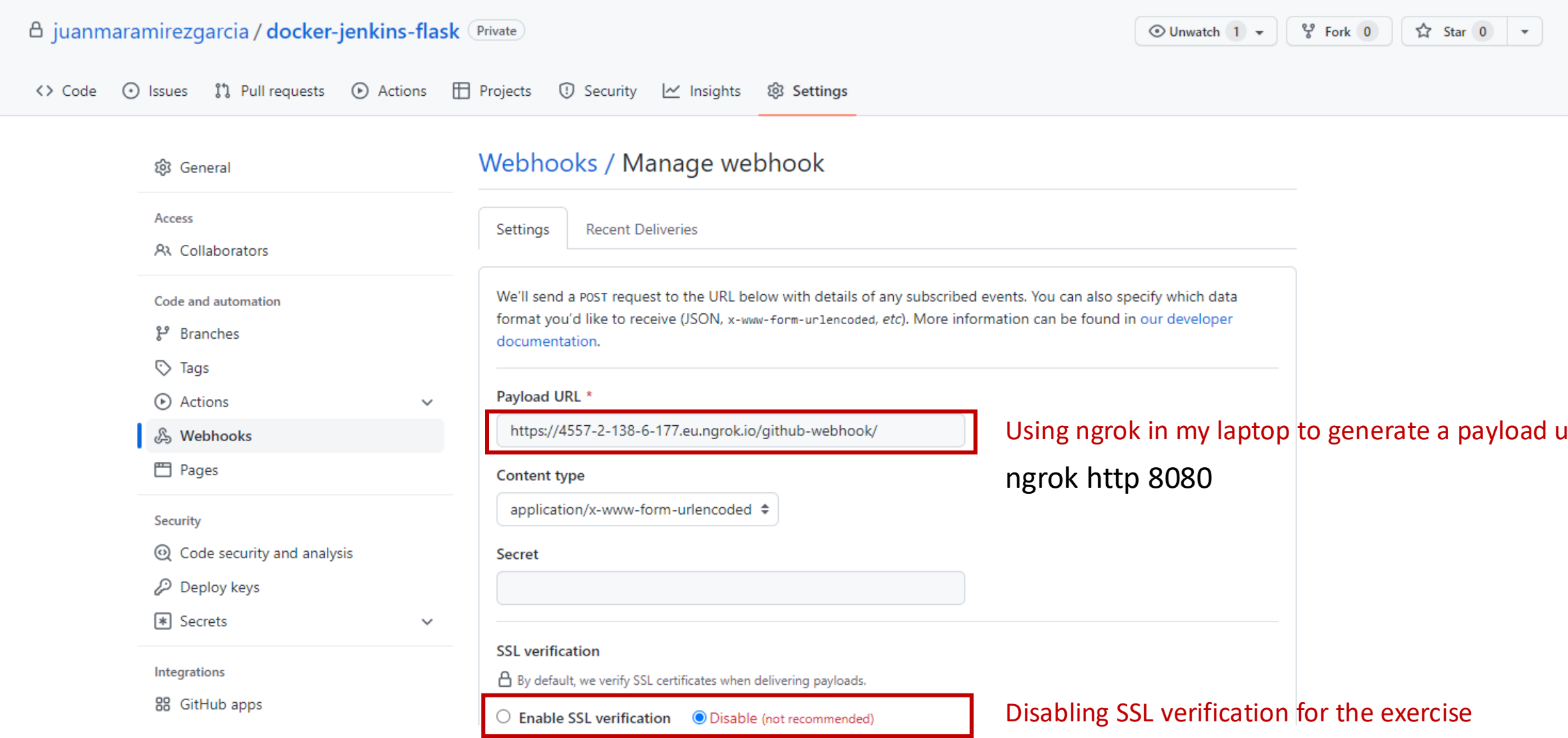
```
docker container stop mycont  
docker container rm mycont  
docker build --pull -t myapp .  
start /b docker run -p 5002:5000 --name mycont myapp
```

[See the list of available environment variables](#)

Save Apply Advanced...

Build Commands:
Be careful because this are
Windows 10 command line

Configuring Webhook in GitHub



The screenshot shows the GitHub repository settings for 'juanmaramirezgarcia / docker-jenkins-flask'. The 'Settings' tab is selected, and the 'Webhooks' section is highlighted in the left sidebar. The 'Webhooks / Manage webhook' page is displayed, showing the 'Settings' tab. The 'Payload URL' is set to 'https://4557-2-138-6-177.eu.ngrok.io/github-webhook/'. The 'Content type' is set to 'application/x-www-form-urlencoded'. The 'Secret' field is empty. The 'SSL verification' section shows 'By default, we verify SSL certificates when delivering payloads.' and the 'Disable (not recommended)' option is selected.

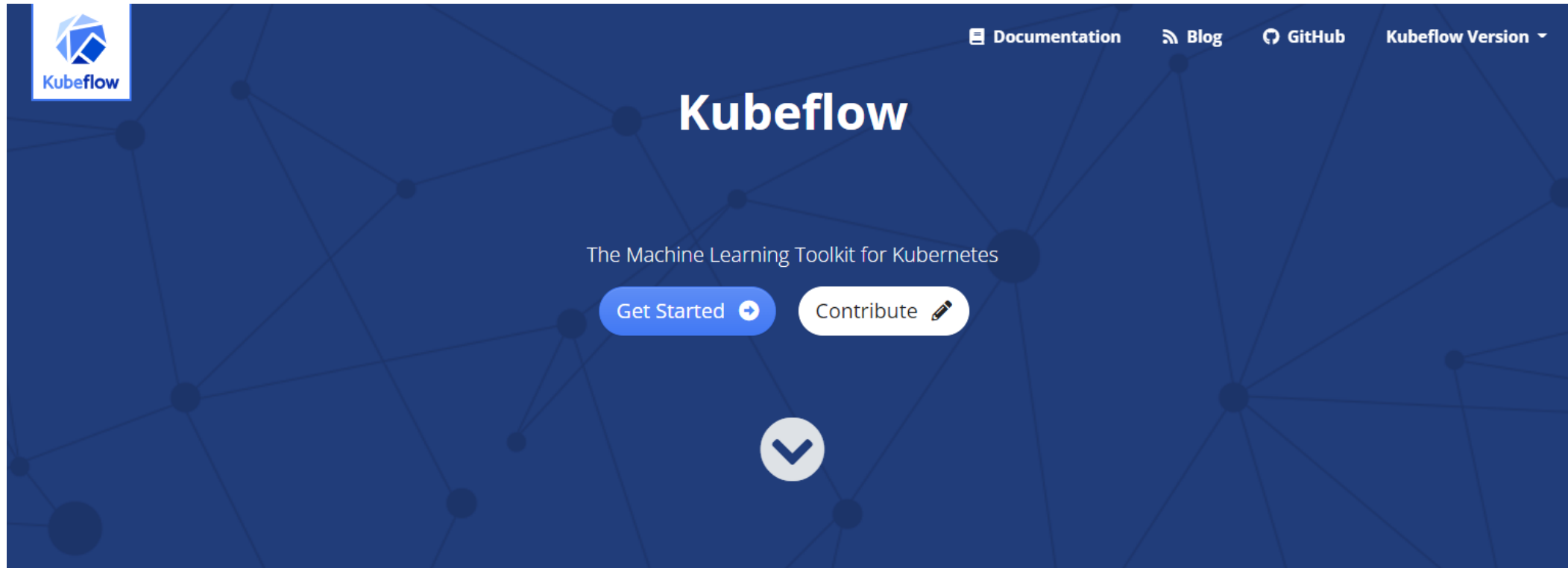
Using ngrok in my laptop to generate a payload url
ngrok http 8080

Disabling SSL verification for the exercise

Kubeflow

What is Kubeflow?

<https://www.kubeflow.org/>



What is Kubeflow?

The Kubeflow project is dedicated to making deployments of machine learning (ML) workflows on Kubernetes simple, portable and scalable. Our goal is not to recreate other services, but to provide a straightforward way to deploy best-of-breed open-source systems for ML to diverse infrastructures. Anywhere you are running Kubernetes, you should be able to run Kubeflow.

What is Kubeflow Pipelines

<https://www.kubeflow.org/docs/components/pipelines/introduction/>

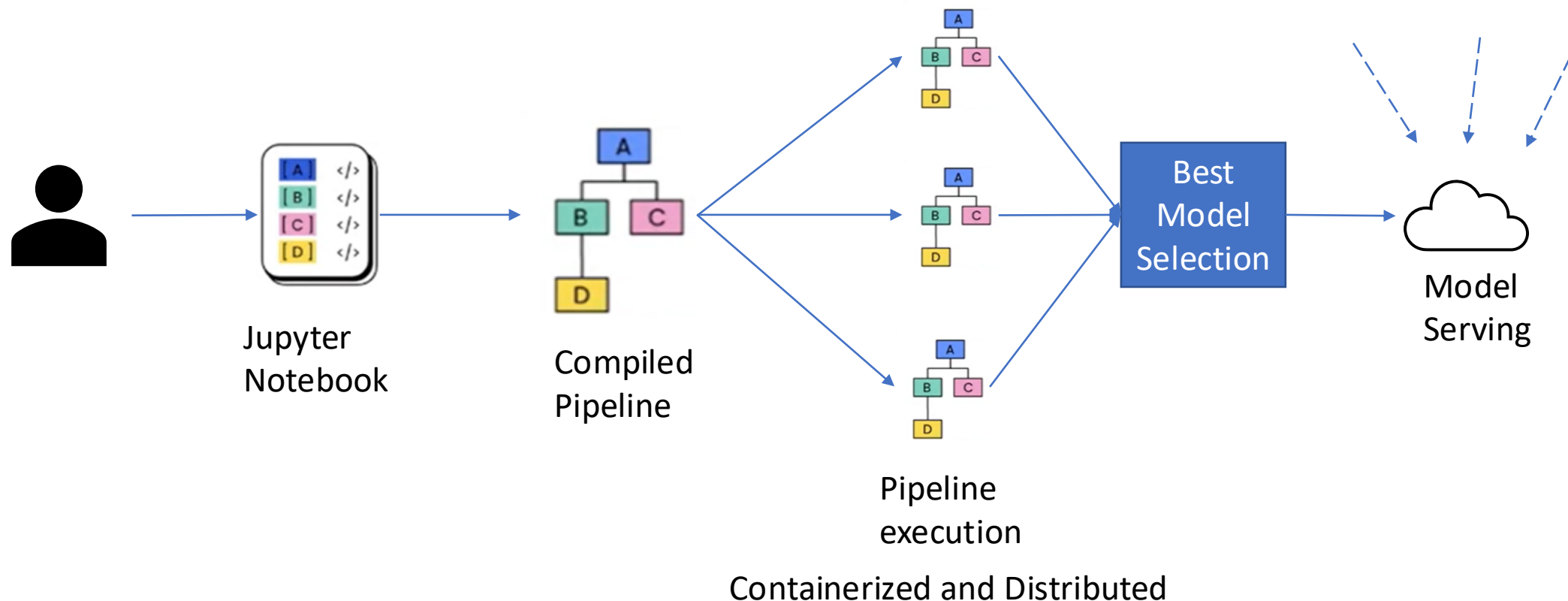
- The Kubeflow Pipelines platform consists of:
 - A user interface (UI) for managing and tracking experiments, jobs, and runs.
 - An engine for scheduling multi-step ML workflows.
 - An SDK for defining and manipulating pipelines and components.
 - Notebooks for interacting with the system using the SDK.
- The following are the goals of Kubeflow Pipelines:
 - End-to-end orchestration: enabling and simplifying the orchestration of machine learning pipelines.
 - Easy experimentation: making it easy for you to try numerous ideas and techniques and manage your various trials/experiments.
 - Easy re-use: enabling you to re-use components and pipelines to quickly create end-to-end solutions without having to rebuild each time.
 - Kubeflow Pipelines is available as a core component of Kubeflow or as a standalone installation.

How to use Kubeflow

- User Interface (UI)
- Command Lines
 - kubectl CLI -> Kubernetes Command Line
 - kfctl CLI -> Kubeflow Command Line
- APIs and SDKs
 - Pipelines SDK

Workflow

<https://github.com/juanmaramirezgarcia/kubeflow-pipelines>



Note: I created another example to deploy in a local Kubeflow installation

https://github.com/juanmaramirezgarcia/kubeflow_pipeline_local.git

Benefits of running a Notebook as a Pipeline



- The steps of the workflow are clearly defined
- Parallelization & Isolation
 - Perfect for hyperparameter tuning
- Data versioning
- Different infrastructure requirements
 - Different hardware (CPU / GPU)

Before

Write your ML code

Create Docker images

Write DSL KFP code

Write your ML code

Create Docker images

Write DSL KFP code

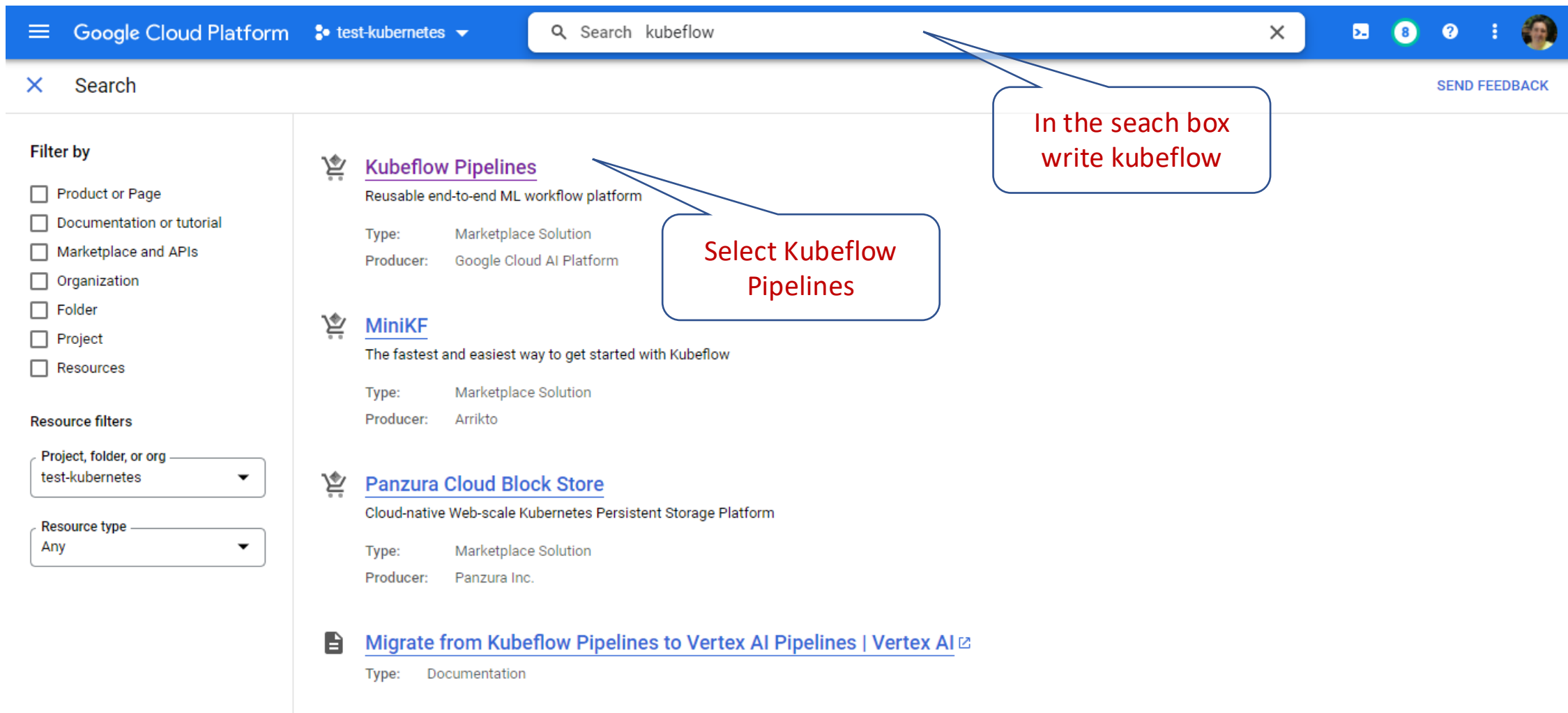
After

Write your ML code

Tag your Notebook cells

Run the pipeline

Create Kubeflow Pipelines Environment

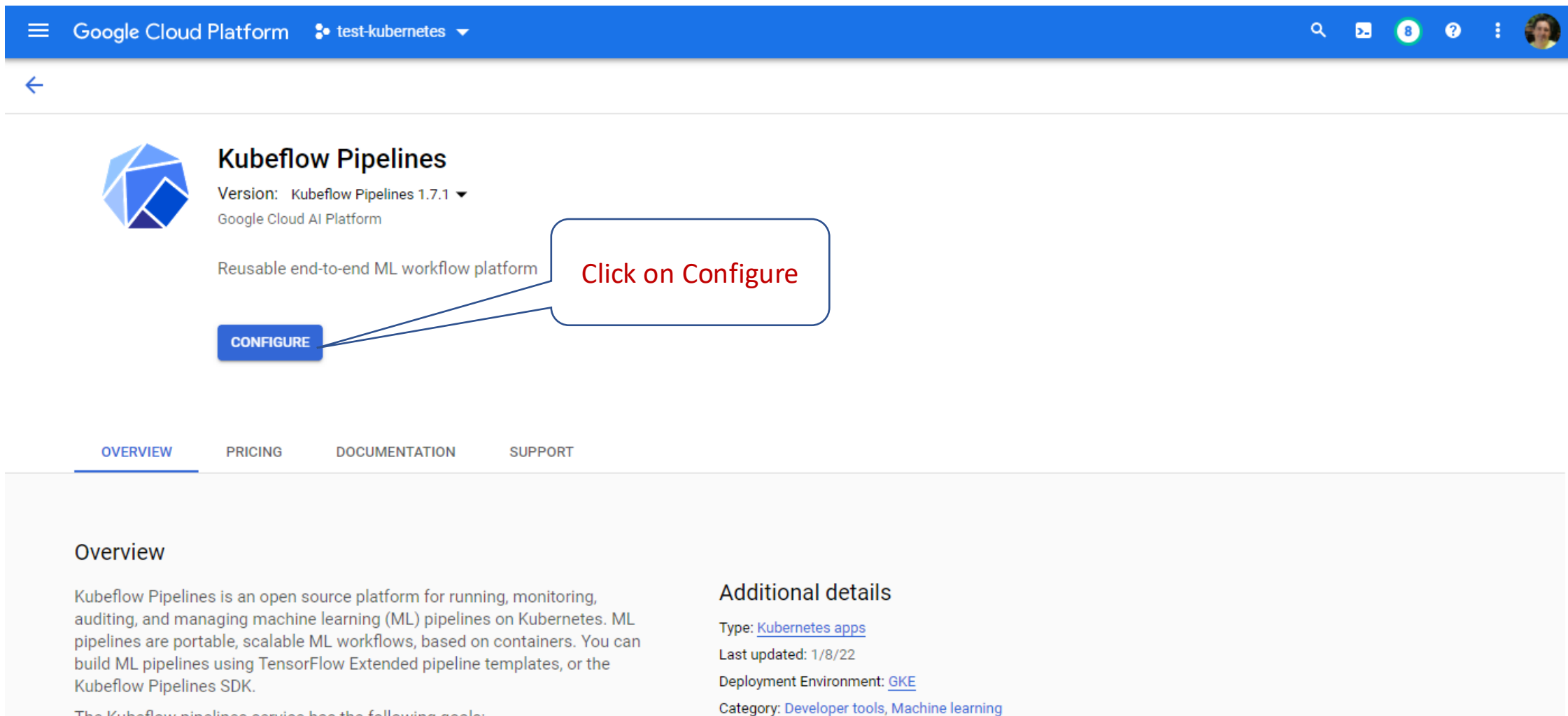


The screenshot shows the Google Cloud Platform search interface. The top navigation bar includes the Google Cloud Platform logo, a dropdown menu for 'test-kubernetes', a search bar containing 'kubeflow', and user profile icons. The search results are displayed on the left side of the page, with a 'Filter by' section on the far left. The search results list several items:

- Kubeflow Pipelines**: Reusable end-to-end ML workflow platform. Type: Marketplace Solution. Producer: Google Cloud AI Platform. An annotation points to this item with the text 'Select Kubeflow Pipelines'.
- MiniKF**: The fastest and easiest way to get started with Kubeflow. Type: Marketplace Solution. Producer: Arrikto.
- Panzura Cloud Block Store**: Cloud-native Web-scale Kubernetes Persistent Storage Platform. Type: Marketplace Solution. Producer: Panzura Inc.
- Migrate from Kubeflow Pipelines to Vertex AI Pipelines | Vertex AI**: Documentation. Type: Documentation.

On the left side, the 'Filter by' section includes checkboxes for 'Product or Page', 'Documentation or tutorial', 'Marketplace and APIs', 'Organization', 'Folder', 'Project', and 'Resources'. The 'Resource filters' section includes a dropdown for 'Project, folder, or org' set to 'test-kubernetes' and a dropdown for 'Resource type' set to 'Any'. An annotation points to the search bar with the text 'In the search box write kubeflow'.

Kubeflow Pipelines Configure



Google Cloud Platform test-kubernetes

Kubeflow Pipelines

Version: Kubeflow Pipelines 1.7.1

Google Cloud AI Platform

Reusable end-to-end ML workflow platform

CONFIGURE

Click on Configure

OVERVIEW PRICING DOCUMENTATION SUPPORT

Overview

Kubeflow Pipelines is an open source platform for running, monitoring, auditing, and managing machine learning (ML) pipelines on Kubernetes. ML pipelines are portable, scalable ML workflows, based on containers. You can build ML pipelines using TensorFlow Extended pipeline templates, or the Kubeflow Pipelines SDK.

The Kubeflow pipelines service has the following goals:

Additional details

Type: [Kubernetes apps](#)

Last updated: 1/8/22

Deployment Environment: [GKE](#)

Category: [Developer tools](#), [Machine learning](#)

Deploy Kubeflow Pipelines

Google Cloud Platform

test-kubernetes

8

Deploy Kubeflow Pipelines

Your app will use compute instances managed in a logical grouping called a "cluster", which will be configured in a way that's great for getting started with Kubernetes. For more options, visit the Kubernetes engine [cluster creation page](#).

Zone

europa-west1-b

?

Network

default

?

Subnetwork

default

?

☒ Allow access to the following Cloud APIs

?

<https://www.googleapis.com/auth/cloud-platform>

CREATE NEW CLUSTER

OR SELECT AN EXISTING CLUSTER

Namespace

default

?

The namespace in which to deploy the application

App instance name *

kubeflow-pipelines-juan

?



Kubeflow Pipelines Overview

Solution provided by Google Cloud AI Platform

Pricing

Note: There is no usage fee for this product. Charges will apply for the use of Google Kubernetes Engine. Please refer to [GCP Price List](#) for the latest pricing.

All products are priced in USD and charged in the currency () specified by your Billing Account. The price for this month is calculated with an exchange rate of 1 USD = 0

Documentation

- [Setting up AI Platform Pipelines](#)
- [Upgrade Notes](#)
- [AI Platform Pipelines Documentation](#)
- [Kubeflow Pipelines Documentation](#)
- [APISpecs](#)
- [Kubeflow Pipelines Github Repo](#)

Create New Cluster or
Select an Existing Cluster

Give a name to the
App instance

Once created go to the
bottom of the page

Deploy Kubeflow Pipelines

Google Cloud Platform

test-kubernetes

8

?



Deploy Kubeflow Pipelines

Cloud SQL instance connection name (Managed storage only) ?

Database username (Managed storage only)
root ?

Database password (Managed storage only) ?

If you are deploying Kubeflow Pipelines with managed storage, specify the database password for Kubeflow Pipelines to use when connecting to your MySQL instance on Cloud SQL. If you leave this field empty, Kubeflow Pipelines connects to your Cloud SQL instance without providing a password. This will fail if a password is required for the username you specified.

Database name prefix (Managed storage only) ?

i

This app has permission to modify resources at the cluster scope.

SHOW MORE

☒ I accept the [GCP Marketplace Terms of Service](#).

DEPLOY

- [AI Platform Pipelines Console](#)
- [AI Platform Pipelines Support](#)
- [Kubeflow Pipelines Github Repo](#)

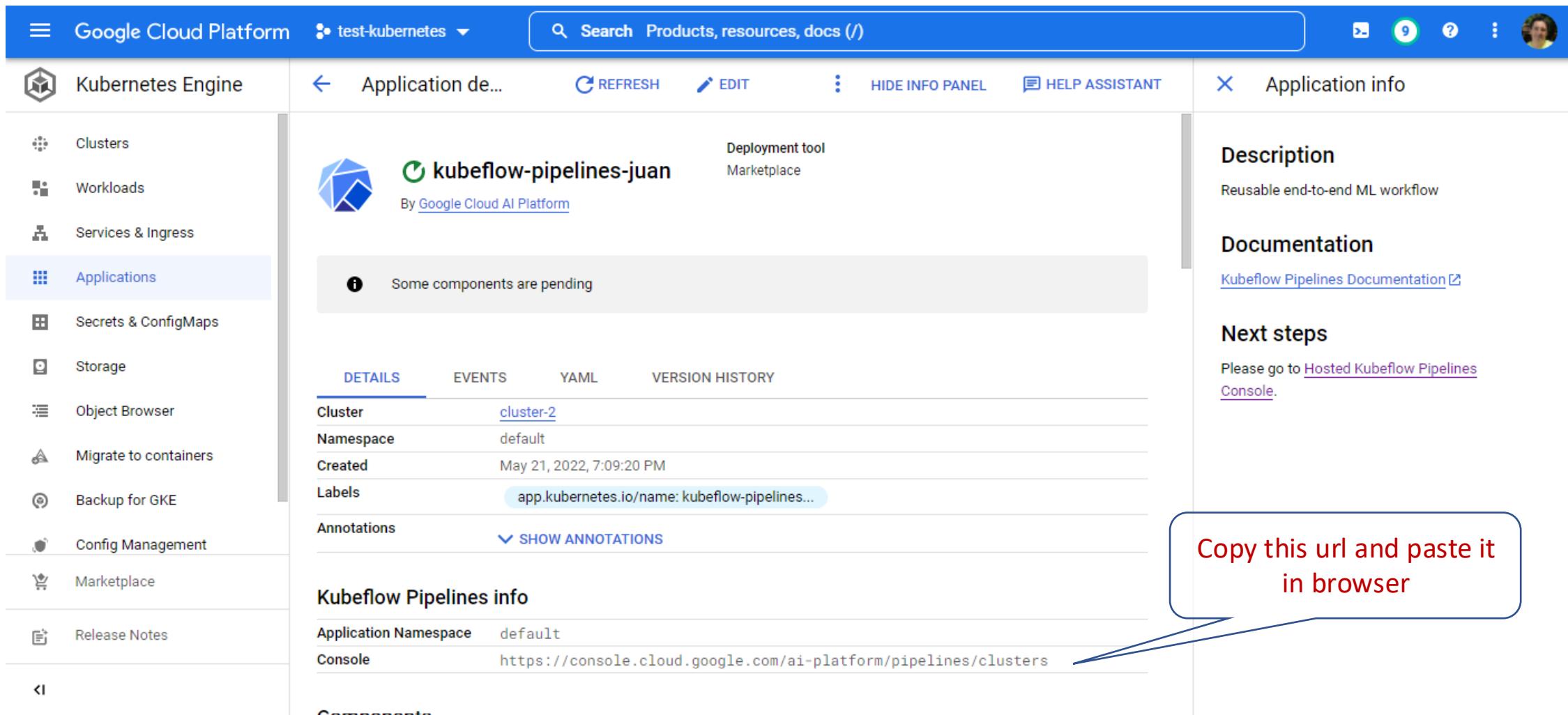
Terms of Service

By deploying the software or accessing the service you are agreeing to comply with the [Google Cloud AI Platform terms of service](#), [GCP Marketplace terms of service](#) and the terms of applicable open source software licenses bundled with the software or service. Please review these terms and licenses carefully for details about any obligations you may have related to the software or service. To the limited extent an open source software license related to the software or service expressly supersedes the GCP Marketplace Terms of Service, that open source software license governs your use of that software or service.

By using this product, you understand that certain account and usage information may be shared with Google Cloud AI Platform for the purposes of financial accounting, sales attribution, performance analysis, and support. ?

Google is providing this software or service "as-is" and any support for this software or service will be provided by Google Cloud AI Platform under their terms of service.

Kubeflow Pipelines Deployed



The screenshot shows the Google Cloud Platform console interface. The left sidebar contains navigation links for Kubernetes Engine, Clusters, Workloads, Services & Ingress, Applications (selected), Secrets & ConfigMaps, Storage, Object Browser, Migrate to containers, Backup for GKE, Config Management, Marketplace, and Release Notes. The main content area displays the 'Application de...' page for 'kubeflow-pipelines-juan'. A message states 'Some components are pending'. Below this, a table provides details for the deployment:

DETAILS	EVENTS	YAML	VERSION HISTORY
Cluster	cluster-2		
Namespace	default		
Created	May 21, 2022, 7:09:20 PM		
Labels	app.kubernetes.io/name: kubeflow-pipelines...		
Annotations	SHOW ANNOTATIONS		

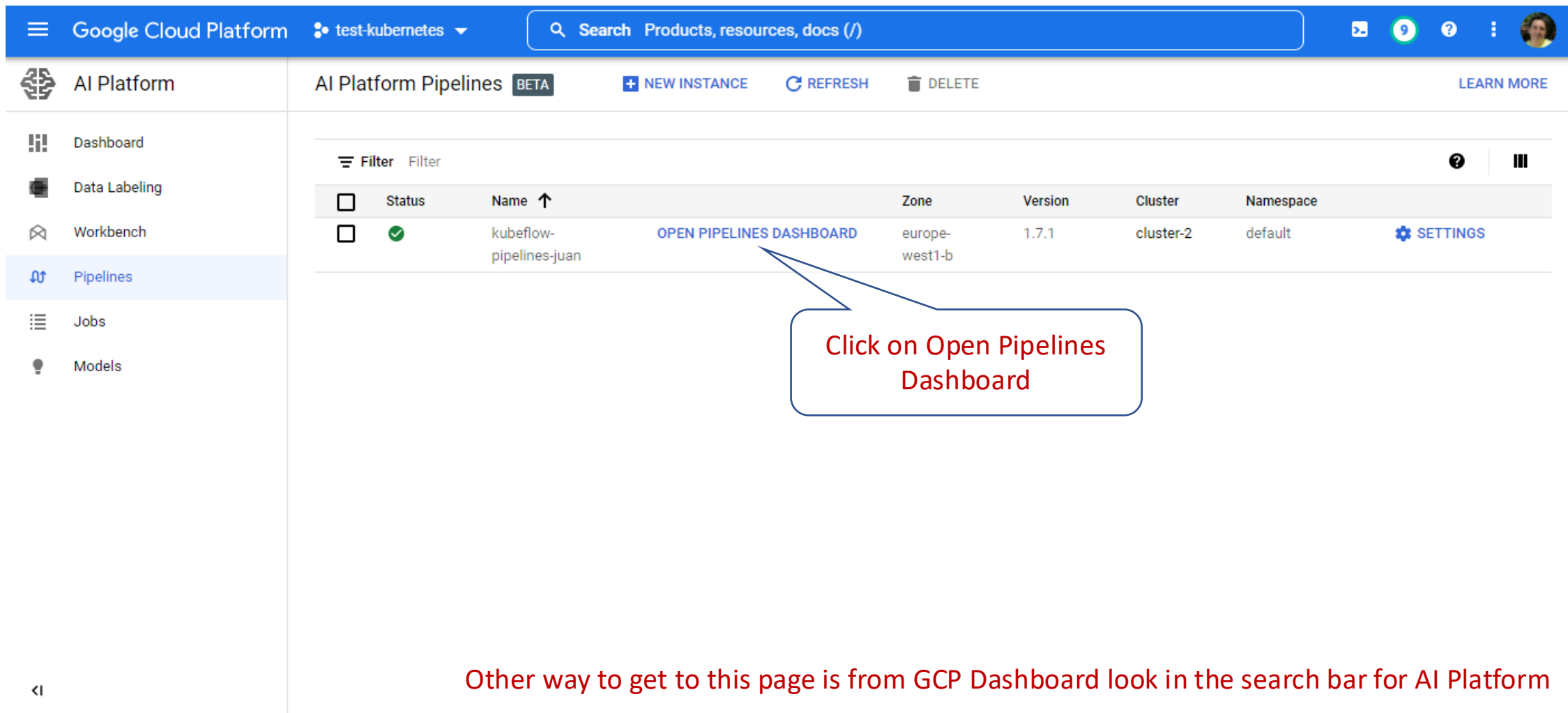
Below the table, the 'Kubeflow Pipelines info' section is visible:

Application Namespace	default
Console	https://console.cloud.google.com/ai-platform/pipelines/clusters

A callout box points to the console URL with the text: 'Copy this url and paste it in browser'.

On the right side of the console, the 'Application info' panel is open, showing the 'Description' (Reusable end-to-end ML workflow), 'Documentation' (Kubeflow Pipelines Documentation), and 'Next steps' (Please go to Hosted Kubeflow Pipelines Console).

AI Platform - Pipelines



Google Cloud Platform test-kubernetes Search Products, resources, docs (/)

AI Platform AI Platform Pipelines **BETA** + NEW INSTANCE REFRESH DELETE LEARN MORE

Filter Filter ?

☐	Status	Name ↑	Zone	Version	Cluster	Namespace	
☐	✓	kubeflow-pipelines-juan	europa-west1-b	1.7.1	cluster-2	default	OPEN PIPELINES DASHBOARD SETTINGS

Click on Open Pipelines Dashboard

Other way to get to this page is from GCP Dashboard look in the search bar for AI Platform

Getting Started with Kubeflow Pipelines

 **Getting Started**

 Pipelines

 Experiments

 Runs

 Recurring Runs

 Artifacts

 Executions

 Documentation 

 Github Repo 



Getting Started

Build your own pipeline with

Kubeflow Pipelines [SDK](#)

Try out [the new Kubeflow Pipelines SDK v2 \(Beta\)](#)

TensorFlow Extended (TFX) [SDK](#) with end-to-end ML Pipeline Template ([Open a Cloud AI Platform Notebook on Google Cloud](#))

Demonstrations and Tutorials

This section contains demo and tutorial pipelines.

Demos - Try an end-to-end demonstration pipeline.

[TFX pipeline demo with Estimator](#)

Classification pipeline with model analysis, based on a public BigQuery dataset of taxicab trips. [source code](#)

[XGBoost Pipeline demo](#)

An example of end-to-end iterative XGBoost model training. [source code](#)

Tutorials - Learn pipeline concepts by following a tutorial.

[V2 Python function based components](#)

Shows different component input and output options for KFP v2 components. [source code](#)

How to create KubeFlow Pipeline

Define component

Info to create the container

Build everything into a function
declaring Output that are going
to be used in next step

```
@component(
    # this component builds an xgboost classifier with xgboost
    packages_to_install=["google-cloud-bigquery", "xgboost", "pandas", "sklearn", "joblib", "pyarrow", "db-dtypes"],
    base_image="python:3.9",
    output_component_file="output_component/create_xgb_model_xgboost.yaml"
)

def build_xgb_xgboost(project_id: str,
                      data_set_id: str,
                      training_view: str,
                      metrics: Output[Metrics],
                      model: Output[Model]
):
    from google.cloud import bigquery
    import xgboost as xgb
    from xgboost import XGBClassifier
    from sklearn.model_selection import train_test_split, StratifiedKFold, RandomizedSearchCV, GridSearchCV
    from sklearn.metrics import accuracy_score, roc_auc_score, precision_recall_curve
    from joblib import dump
    import pandas as pd
    import pyarrow

    client = bigquery.Client(project=project_id)

    data_set = f"{project_id}.{data_set_id}.{training_view}"
    build_df_for_xgboost = '''
    SELECT * FROM `{data_set}`
    '''.format(data_set = data_set)

    job_config = bigquery.QueryJobConfig()
    df = client.query(build_df_for_xgboost, job_config=job_config).to_dataframe() # Make an API request.
    df = pd.get_dummies(df.drop(['fullVisitorId'], axis=1), prefix=['visited_dma', 'visited_daypart', 'visited_dow'])
    df = df.dropna()

    X = df.drop(['label'], axis=1).values
    y = df['label'].values

    X_train, X_test, y_train, y_test = train_test_split(X,y)
    train = xgb.DMatrix(X_train, label=y_train)
    test = xgb.DMatrix(X_test, label=y_test)
```

How to create Kubeflow Pipeline

Define pipeline component

Define all the calls to steps in the pipeline

```
@dsl.pipeline(  
    # Default pipeline root. You can override it when submitting the pipeline.  
    pipeline_root=PIPELINE_ROOT,  
    # A name for the pipeline.  
    name="pipeline-test",  
    description='Propensity BigQuery ML Test'  
)  
def pipeline():  
    create_input_view_op = create_input_view(view_name = VIEW_NAME,  
                                              data_set_id = DATA_SET_ID,  
                                              project_id = PROJECT_ID,  
                                              bucket_name = BUCKET_NAME,  
                                              blob_path = BLOB_PATH  
                                              )  
  
    build_bqml_logistic_op = build_bqml_logistic(project_id = PROJECT_ID,  
                                                  data_set_id = DATA_SET_ID,  
                                                  model_name = 'bqml_logistic_model',  
                                                  training_view = VIEW_NAME  
                                                  )  
  
    build_bqml_xgboost_op = build_bqml_xgboost(project_id = PROJECT_ID,  
                                                data_set_id = DATA_SET_ID,  
                                                model_name = 'bqml_xgboost_model',  
                                                training_view = VIEW_NAME  
                                                )  
  
    build_bqml_automl_op = build_bqml_automl (project_id = PROJECT_ID,  
                                              data_set_id = DATA_SET_ID,  
                                              model_name = 'bqml_automl_model',  
                                              training_view = VIEW_NAME  
                                              )
```

How to create Kubeflow Pipeline

```
build_xgb_xgboost_op = build_xgb_xgboost(project_id = PROJECT_ID,
                                         data_set_id = DATA_SET_ID,
                                         training_view = VIEW_NAME
                                         )

evaluate_bqml_logistic_op = evaluate_bqml_logistic(project_id = PROJECT_ID,
                                                    data_set_id = DATA_SET_ID,
                                                    model_name = 'bqml_logistic_model',
                                                    training_view = VIEW_NAME
                                                    )

evaluate_bqml_xgboost_op = evaluate_bqml_xgboost(project_id = PROJECT_ID,
                                                  data_set_id = DATA_SET_ID,
                                                  model_name = 'bqml_xgboost_model',
                                                  training_view = VIEW_NAME
                                                  )

evaluate_bqml_automl_op = evaluate_bqml_automl(project_id = PROJECT_ID,
                                                data_set_id = DATA_SET_ID,
                                                model_name = 'bqml_automl_model',
                                                training_view = VIEW_NAME
                                                )

deploy_xgb_op = deploy_xgb(project_id = PROJECT_ID,
                           model=build_xgb_xgboost_op.outputs["model"]
                           )
```

Define the order of steps

```
build_bqml_logistic_op.after(create_input_view_op)
build_bqml_xgboost_op.after(create_input_view_op)
build_bqml_automl_op.after(create_input_view_op)
build_xgb_xgboost_op.after(create_input_view_op)

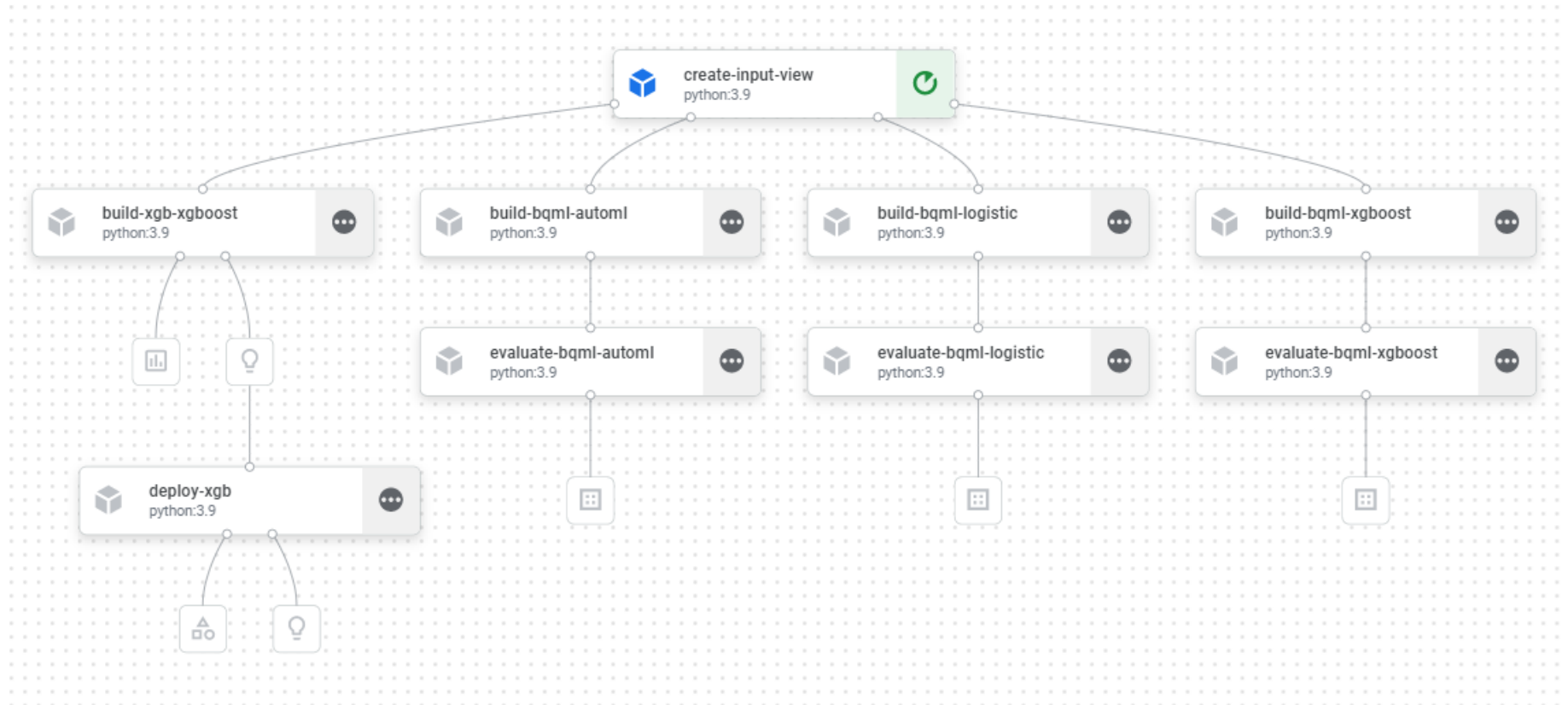
evaluate_bqml_logistic_op.after(build_bqml_logistic_op)
evaluate_bqml_xgboost_op.after(build_bqml_xgboost_op)
evaluate_bqml_automl_op.after(build_bqml_automl_op)

deploy_xgb_op.after(build_xgb_xgboost_op)
```

Compile

```
compiler.Compiler().compile(
    pipeline_func=pipeline, package_path="pipeline.json"
)
```

Kubeflow Pipeline



Apache Airflow

Apache Airflow



<https://airflow.apache.org/>

Apache Airflow

Airflow is a platform created by the community to programmatically author, schedule and monitor workflows.

Principles



Scalable

Airflow has a modular architecture and uses a message queue to orchestrate an arbitrary number of workers. Airflow is ready to scale to infinity.



Dynamic

Airflow pipelines are defined in Python, allowing for dynamic pipeline generation. This allows for writing code that instantiates pipelines dynamically.



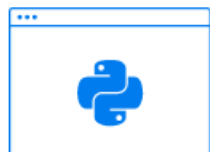
Extensible

Easily define your own operators and extend libraries to fit the level of abstraction that suits your environment.



Elegant

Airflow pipelines are lean and explicit. Parametrization is built into its core using the powerful Jinja templating engine.



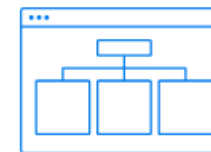
Pure Python

No more command-line or XML black-magic! Use standard Python features to create your workflows, including date time formats for scheduling and loops to dynamically generate tasks. This allows you to maintain full flexibility when building your workflows.



Useful UI

Monitor, schedule and manage your workflows via a robust and modern web application. No need to learn old, cron-like interfaces. You always have full insight into the status and logs of completed and ongoing tasks.



Robust Integrations

Airflow provides many plug-and-play operators that are ready to execute your tasks on Google Cloud Platform, Amazon Web Services, Microsoft Azure and many other third-party services. This makes Airflow easy to apply to current infrastructure and extend to next-gen technologies.



Easy to Use

Anyone with Python knowledge can deploy a workflow. Apache Airflow does not limit the scope of your pipelines; you can use it to build ML models, transfer data, manage your infrastructure, and more.



Open Source

Wherever you want to share your improvement you can do this by opening a PR. It's simple as that, no barriers, no prolonged procedures. Airflow has many active users who willingly share their experiences. Have any questions? Check out our buzzing slack.

DAG Creation

Imports needed from airflow

```
1 from airflow import DAG
2 from airflow.operators.python import PythonOperator, BranchPythonOperator
3 from airflow.operators.bash import BashOperator
```

Xcom is the way to share results between tasks

```
5 from random import randint
6 from datetime import datetime
7
8 def _choose_best_model(ti):
9     accuracies = ti.xcom_pull(task_ids=[
10         'training_model_A',
11         'training_model_B',
12         'training_model_C'
13     ])
14     best_accuracy = max(accuracies)
15     if (best_accuracy > 8):
16         return 'accurate'
17     return 'inaccurate'
```

The fake training_model function simply returns a random integer between 1 and 10

```
19
20 def _training_model():
21     return randint(1, 10)
```

Create DAG
With start date and scheduling interval

```
22
23 with DAG("my_dag", start_date=datetime(2021, 1, 1),
24         schedule_interval="@daily", catchup=False) as dag:
```

The task training_model_A is a PythonOperator that will call the training_model function and has task_id "training_model_A"

```
25
26     training_model_A = PythonOperator(
27         task_id="training_model_A",
28         python_callable=_training_model
29     )
```

DAG Creation cont.

Imports needed from airflow

The choose_best_model task is
BranchPythonOperator

The final tasks 'accurate' and 'inaccurate'
are BashOperator that simply writes a text
string as a result

```
30
31  training_model_B = PythonOperator(
32      task_id="training_model_B",
33      python_callable=_training_model
34  )
35
36  training_model_C = PythonOperator(
37      task_id="training_model_C",
38      python_callable=_training_model
39  )
40
41  choose_best_model = BranchPythonOperator(
42      task_id="choose_best_model",
43      python_callable=_choose_best_model
44  )
45
46  accurate = BashOperator(
47      task_id="accurate",
48      bash_command="echo 'accurate'"
49  )
50
51  inaccurate = BashOperator(
52      task_id="inaccurate",
53      bash_command="echo 'inaccurate'"
54  )
55
56  [training_model_A, training_model_B, training_model_C] >> choose_best_model >> [accurate, inaccurate]
```

This indicates the order of the tasks in the DAG

Airflow UI

Airflow UI interface showing a DAG (Directed Acyclic Graph) visualization.

Navigation tabs: DAGs, Security, Browse, Admin, Docs. Time: 18:04 UTC. User: AA.

View options: Grid, Graph (selected), Calendar, Task Duration, Task Tries, Landing Times, Gantt, Details, Code. Audit Log.

Filters: Date (2022-05-29T17:46:04Z), Runs (25), Run (manual__2022-05-29T17:46:03.861194+00:00). Find Task... button.

Layout: Left > Right. Update button.

Operator types: BashOperator, BranchPythonOperator, PythonOperator.

Status legend: queued, running, success, failed, up_for_retry, up_for_reschedule, upstream_failed, skipped, scheduled, deferred, no_status.

Auto-refresh toggle: Auto-refresh (disabled).

DAG visualization (Graph view):

```
graph LR; training_model_A[training_model_A] --> choose_best_model[choose_best_model]; training_model_B[training_model_B] --> choose_best_model; training_model_C[training_model_C] --> choose_best_model; choose_best_model --> accurate[accurate]; choose_best_model --> inaccurate[inaccurate];
```